

TP N° 2

Initiation à R : Structures de contrôle, vecteurs et matrices

But du TP :

Utiliser les structures de contrôle et les vecteurs et matrices

Énoncé :

A. Structures alternatives

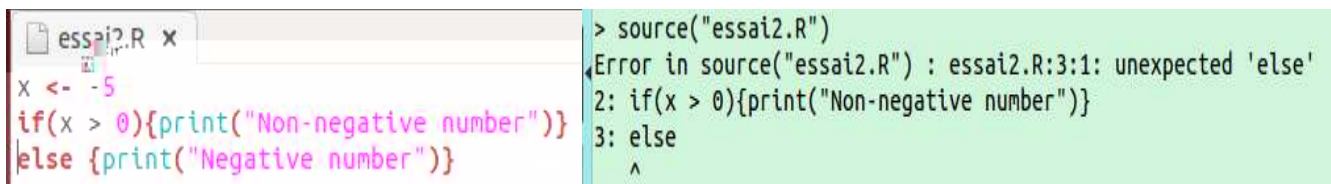
Illustrer les structures alternatives (if, else, else if) et les vecteurs (c(), rep(), seq())

```
1 x <- -5
2 if(x > 0){
3   print("Non-negative number")
4 } else {
5   print("Negative number")
6 }
```

Tester les structures alternatives et les vecteurs

Utiliser les structures alternatives et les vecteurs pour créer des données et les manipuler

Illustrer les structures alternatives et les vecteurs



```
essai2.R x
x <- -5
if(x > 0){print("Non-negative number")}
else {print("Negative number")}

> source("essai2.R")
Error in source("essai2.R") : essai2.R:3:1: unexpected 'else'
2: if(x > 0){print("Non-negative number")}
3: else
  ^
```

Illustrer les structures alternatives et les vecteurs

```
1 x <- 5; y <- 1
2 if(y){
3   if(x>0){cat("positif")}
4   else {cat("negatif ou nul")}
5 }
```

- Illustrer les structures alternatives et les vecteurs

B. Appels de fonctions R

Utiliser les fonctions R pour manipuler les données et les structures de contrôle

Exemple :

```
writeLines("146.6", "popRate.txt")
```

```
> writeLines("146.6", "popRate.txt")
```

```
> writeLines(text="146.6", con="popRate.txt")
```

```
> writeLines(con="popRate.txt", text="146.6")
```

```
> writeLines("146.6")
```

146.6

Exemple 2:

- `seq(from=1, by=5, to=20)`
- Utilis `?seq`

C. Vecteurs

Function	Example	Result
----------	---------	--------

- `seq(1, 5, 20)`
- `seq(2, 15, 3) ; a[4]`
- `a <- seq(2, 25, 3) ; a[3:6]`
- `a[c(2, 2, 1, 1)]`

```
> a[seq(2,10,3)]
```

```

      _____ ti      i tilis      s      t      s t      tl s t      st
                    ifelse(<boolVect>, x, y)
l      t      it t      t      l i      t      t t      i t      t      t t      tl
l      t      st      t      l      l      t      t      st      s      s
l      ts      s l      l      t s it      U      s      t      s      t      s t
t      s si      ss i

```

Example :

```

> a = c(5,7,2,9)
> ifelse(a %% 2 == 0,"even","odd")
[1] "odd" "odd" "even" "odd"

```

```

•      i      s i t      i t t
      l      s      s l      s ss i s

```

D. Matrices

```

U      ti      s      st      l      ti      ma
> x <- c(2, 4, 3, 1, 5, 7)
> A = matrix(
+   x,          # the data elements
+   nrow=2,     # number of rows
+   ncol=3,     # number of columns
+   byrow = TRUE) # fill matrix by rows

```

```

i it      ti      x      li li      li      ti      t      i      l

```

```

U      l      t      i      i      s i      t t      l      ssi      i      i
      i      li      t t      t it      i      i      l      i      li
      l      t t      t it      s lt t st      t      i
l      i      l

```

```

s      s      ssi      t i      s      s      ti      t      l      s lt t      l

```

```
> A[,c(1,3)]
```

E. Structures répétitives

```

is      s      s st      t      s      titi      s      s      ll      s      is      t
• for (<variable> in <sequence>) <instruction>
• while (<condition>) <instruction>
• repeat {<instruction> ; if(<condition>)break}
t      l      ss      i      ts      s      i      ts      t      t      l      s      lt      t

```

- Script 1 :

```
mat <- matrix(data = seq(10, 21, by=1), nrow = 6, ncol =2)
for (i in 1:nrow(mat))
  for (j in 1:ncol(mat)){
    cat(c("L'element a ligne ", i, " et la colonne ",j, " est ", mat[i,j]))
    cat("\n")
  }
```

lis s s i t s i t l s l s l s il t t

- Script 2 :

```
mat <- matrix(data = seq(1, 12, by=1), nrow = 6, ncol =2)

vMoyLin<-rep(0,nrow(mat))
for(i in 1:nrow(mat)){
  for(j in 1:ncol(mat)){
    vMoyLin[i]<-vMoyLin[i]+mat[i,j]
  }
  vMoyLin[i]<-vMoyLin[i]/ncol(mat)
}

vMoyCol<-rep(0,ncol(mat))
for(j in 1:ncol(mat)){
  for(i in 1:nrow(mat)){
    vMoyCol[j]<-vMoyCol[j]+mat[i,j]
  }
  vMoyCol[j]<-vMoyCol[j]/nrow(mat)
}
print(mat)
print(vMoyLin)
print(vMoyCol)
```

- Script 3 :

```
mat <- matrix(data = seq(1, 12, by=1), nrow = 6, ncol =2)

for(i in 1:nrow(mat))
  vMoyLin[i]<-mean(mat[i,])

for(j in 1:ncol(mat))
  vMoyCol[j]<-mean(mat[,j])

print(mat)
print(vMoyLin)
print(vMoyCol)
```

F. Fonctions utilisateur

U ti st l i t l function s l l s t

function_name ← function(<arg_1>, <arg_2>, ...) { <Function body> }

t l l s i t

```
1 = mafonction <- function(a,b,c=3) {
2   result <- a * b + c
3   return(result)
4 }
5 # Appeler la fonction en utilisant la position des arguments.
6 mafonction(5,3,11)
7 # Appeler la fonction en utilisant les noms des arguments.
8 mafonction(b = 5, a = 11, c = 3)
9 # Appeler la fonction avec des valeurs d'arguments par défaut.
10 mafonction(10)
```

st l s lt t stl s ls

i l l i iti lis ti l tb t t t

```
> mafonction(10)
```

```
Error in mafonction(10) : argument "b" is missing, with no default
```

l tilis ti s ti s s s t t s t l l ti s lt l l
s s li s til s t isi ilit s t s issi t

G. Manipulations vectorielles avec apply()

is s ti s lis t s i l ti s t i ll s i tt t l
s s t it l tilis ti l s i ll s i apply() st l s it
apply(X,MARGIN,FUN)

- st ti
- st i l i iss t tl ti st li l s
- ll s li li s l s ll s li l s
- U stl ti ss it li s l ts i i t ll
ti is ti i i l tilis t

t l s i ts i t

```
mat <- matrix(data = seq(1, 12, by=1), nrow = 6, ncol = 2)

vMoyLin <- apply(mat, 1, mean)
vMoyCol <- apply(mat, 2, mean)

print(mat)
print(vMoyLin)
print(vMoyCol)
```

l s lt t s s i ts t l s ti E

itl s i ts i t

```
1 mat <- matrix(sample(-100:100,21),nrow = 3)
2 print(mat)
3 myFct <- function(X){
4   ecart <- max(X)-min(X)
5   return(ecart)
6 }
7 print(mat)
8 vEcartLin <- apply(mat,1,FUN=myFct)
9 vEcartCol <- apply(mat,2,FUN=myFct)
10 print(vEcartLin)
11 print(vEcartCol)
```

```
[3,] 59 81 39 29 -88 19 14
[,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,] 0 94 -91 85 37 61 77
[2,] -87 -43 47 -81 90 -51 76
[3,] 59 81 39 29 -88 19 14
[1] 185 177 169
[1] 146 137 138 166 178 112 63
```

- Utilises the following functions
- `sample`
- `myFct`
- `tvEcartLin` `tvEcartCol`

sample {base}

R Documentation

Random Samples and Permutations

Description

`sample` takes a sample of the specified size from the elements of `x` using either with or without replacement.

Usage

```
sample(x, size, replace = FALSE, prob = NULL)
```

```
sample.int(n, size = n, replace = FALSE, prob = NULL,
           useHash = (!replace && is.null(prob) && size <= n/2 && n > 1e7)
```

Arguments

`x` either a vector of one or more elements from which to choose, or a positive integer. See 'Details.'

`n` a positive number, the number of items to choose from. See 'Details.'

`size` a non-negative integer giving the number of items to choose.

`replace` should sampling be with replacement?

`prob` a vector of probability weights for obtaining the elements of the vector being sampled.

```
st <- list()
l <- list()
l <- myFct(1:100)
l <- tvEcartLin(l)
```

```
> myFct(1:100)
```

```
[1] 99
```

```
> rm(list = ls())
```

```
> myFct(1:100)
```

```
Error in myFct(1:100) : could not find function "myFct"
```